

CS 4372

Assignment #1

Linear Regression Analysis

Teammates:

Taylor Mitchell

Dylan Ramdhan

Link: https://github.com/dylanramdhan/CS4372/tree/main/Assignment_1

Works Cited

[UCI ML Repo's Automobile Database](#)

Sci-kit Learn [SGDRegressor Reference](#)

Pandas DataFrame [Histogram Reference](#)

Seaborn [Heatmap Reference](#)

1. Stochastic Gradient Descent (SGDRegression)

1.1. After examining attributes and target variables, we can answer the questions of whether or not the attributes are normally distributed, and if not, what could be the reason.

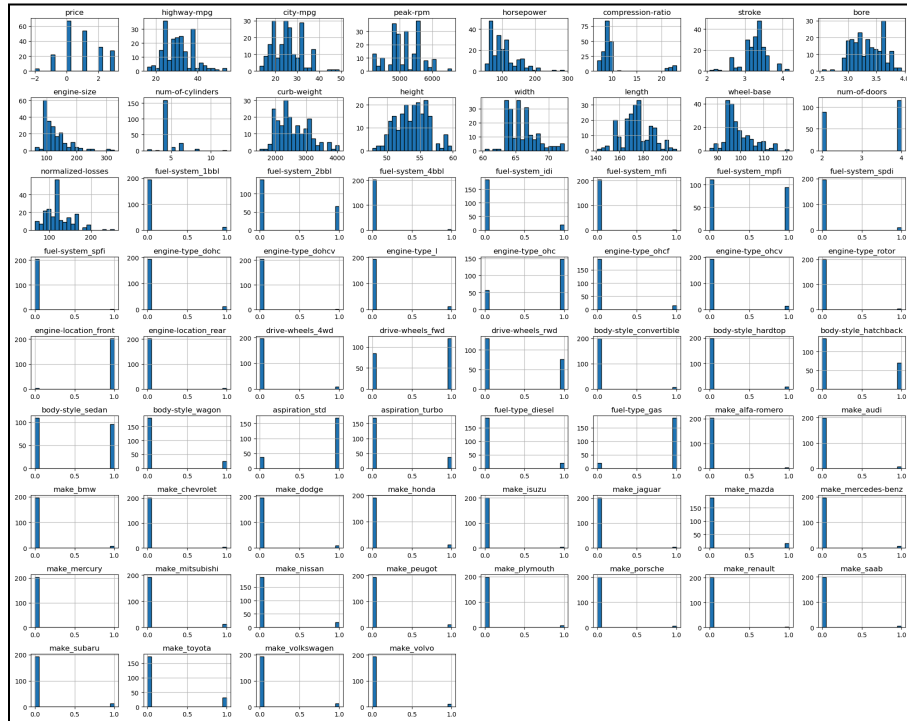


Fig. 1.1.1.

1.1.1. Most attributes are not normally distributed, however, several such as the **price**, **engine-size**, **horsepower** graphs are right-skewed due to a small number of **high-end/high-performance cars**, other graphs such as **highway-mpg** and **city-mpg** graphs are multi-modal because of the dataset mixing distinct vehicle classes, and only a few like the width, wheel-base, height graphs come close to a normal shape.

1.2. In order to find how the attributes that are correlated to each other and the target variable, we performed a numerical and visualization by utilizing a Seaborn heatmap function for the data.

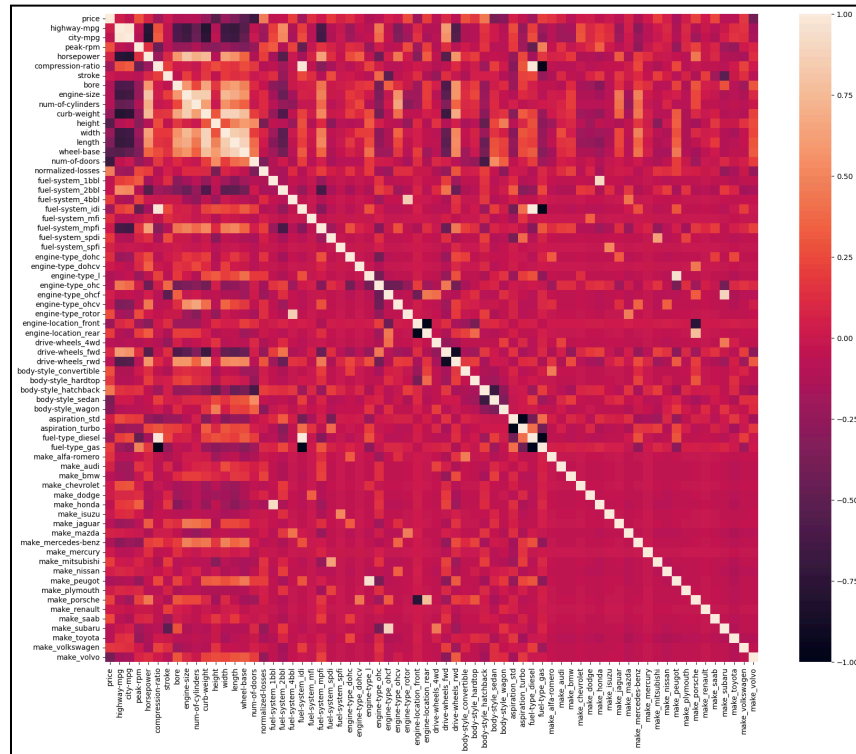


Fig. 1.2.1.

1.2.1. Upon analyzing Fig. 1 heatmap, it appears that the **price** attribute correlates most strongly and positively with **engine-size**, **curb-weight**, **horsepower**, and **width** (bigger, more powerful cars tend to cost more), and negatively with **highway-mpg** and **city-mpg** (more fuel-efficient cars tend to be cheaper, smaller vehicles); the one-hot encoded columns (individual makes, fuel systems, body styles) mostly show weak correlation with price on their own, since brand or configuration alone doesn't drive cost the way size and power do.

1.2.2. Beyond the price row, the strongest correlations in the whole heatmap show up between complementary pairs from the same categorical variables such as the **fuel-type_gas/fuel-type_diesel** or even the **engine-location_front/engine_location_rear** but those are near-perfect negative correlations simply because of how one-hot encoding works (a car can't be both gas and diesel), not a meaningful relationship worth interpreting.

- 1.3. Moving forward into Model Construction for SGD Regressor, this data is the model's **coefficient**, **intercept**, and **test scores** (respectively), which is generated right after fitting the SGDRegressor with scikit-learn's default hyperparameters (*Fig. 1.3.1*). Before beginning to tune, I wanted to see the baseline when comparing it to the tuned version.

```
[-0.01997133 -0.43945026  0.44443646 -0.29892885  0.03133707 -0.28707546]  
[0.86668165]  
0.18742954318259264
```

Fig. 1.3.1.

- 1.3.1. In relation to the data above, it is relative to the **features** (*engine-size*, *curb-weight*, *horsepower*, *width*, *highway-mpg*, and *city-mpg*). The coefficients for *horsepower* (0.444) and *curb-weight* (-0.440) stood out the most to me because they have the largest magnitudes and are pulling in opposite directions. Since the features were standardized before training, each coefficient reflects the effect of a one-standard-deviation change in that feature. *Horsepower's* positive coefficient means cars with more horsepower tend to get a higher (riskier) symboling rating, while *curb-weight's* negative coefficient means heavier cars tend to get a lower (safer) rating. That makes sense as the more powerful a car performs, the more likely it would be rated riskier, while a heavier car feels like it would hold up better structurally, so a safer rating makes sense. *Width* (-0.299) and *city-mpg* (-0.287) also have a noticeable effect, just smaller than those two *engine-size* (-0.020) and *highway-mpg* (0.031), on the other hand, barely move the needle here, which surprised me a little since I expected engine size to matter more.
- 1.4. In addition to the initial tuning, we performed calculating evaluation metrics. Respectively, **Mean Square Error (MSE)**, **Mean Absolute Error (MSE)**, **Explained Variance Score (EVS)**, and **R2 Score**.

```
(1.3177079507937257,  
0.862358120398032,  
0.20438567395053064,  
0.18742954318259264)
```

Fig. 1.4.1.

- 1.4.1. MSE (1.318) and MAE (0.862) both measure prediction error, but they calculate it differently. MSE squares each error before averaging them, so it penalizes larger mistakes more heavily. On the other hand, MAE simply averages the absolute difference between predicted and actual symboling values, without that extra penalty for bigger errors. An MAE of 0.862 means predictions are, on average, under one full rating point away from the true value. That sounds reasonable in absolute terms, but symboling only spans a range of -3 to +3, so an error of nearly a full point represents a meaningful portion of that scale.

1.4.2. As for the explained variance score (0.204), it is close to the R^2 score (0.187).

Since the two are similar rather than far apart, this suggests the model's errors are not being driven by systematic bias. Instead, the model is simply not capturing much of the underlying variance in the target. Together, these metrics reinforce the baseline R^2 result: before tuning, the model explains only a small portion of what drives symboling, which motivates the tuning performed in the following section.

1.5. After initial tuning, we began performing tuning hyperparameters. Prior to model training, all features were standardized using scikit-learn's StandardScaler, converting each attribute to have a mean of 0, and standard deviation of 1. This step is crucial in the *SGDRegressor* model, since gradient-based optimization converges more reliably and efficiently when features are on comparable scales.

	penalty	alpha	l1_ratio	learning_rate	max_iter	loss	train_mse	test_mse	train_r2	test_r2	test_mae	test_evs
0	l1	0.0001	0.15	constant	1000	squared_error	1.331284	1.543011	0.123342	0.048496	0.981289	5.884441e-02
1	l1	0.0001	0.15	constant	1000	huber	1.387130	1.340630	0.086567	0.173294	0.870000	1.754079e-01
2	l1	0.0001	0.15	constant	2000	squared_error	1.201226	1.278211	0.208986	0.211785	0.862168	2.460517e-01
3	l1	0.0001	0.15	constant	2000	huber	1.408500	1.344971	0.072495	0.170618	0.869928	1.739196e-01
4	l1	0.0001	0.15	invscaling	1000	squared_error	1.217595	1.311387	0.198207	0.191328	0.859976	2.086946e-01
...	...	...	...	...	...	...	...	...	...	...	...	...
283	elasticnet	0.1000	0.85	constant	2000	huber	2.015151	1.919917	-0.326988	-0.183925	1.118822	0.000000e+00
284	elasticnet	0.1000	0.85	invscaling	1000	squared_error	1.297165	1.408165	0.145810	0.131649	0.899334	1.476011e-01
285	elasticnet	0.1000	0.85	invscaling	1000	huber	2.153097	2.028727	-0.417826	-0.251023	1.134511	2.220446e-16
286	elasticnet	0.1000	0.85	invscaling	2000	squared_error	1.294252	1.408191	0.147728	0.131633	0.899906	1.477231e-01
287	elasticnet	0.1000	0.85	invscaling	2000	huber	2.153447	2.029007	-0.418057	-0.251196	1.134548	0.000000e+00

288 rows x 12 columns

Fig. 1.5.1.

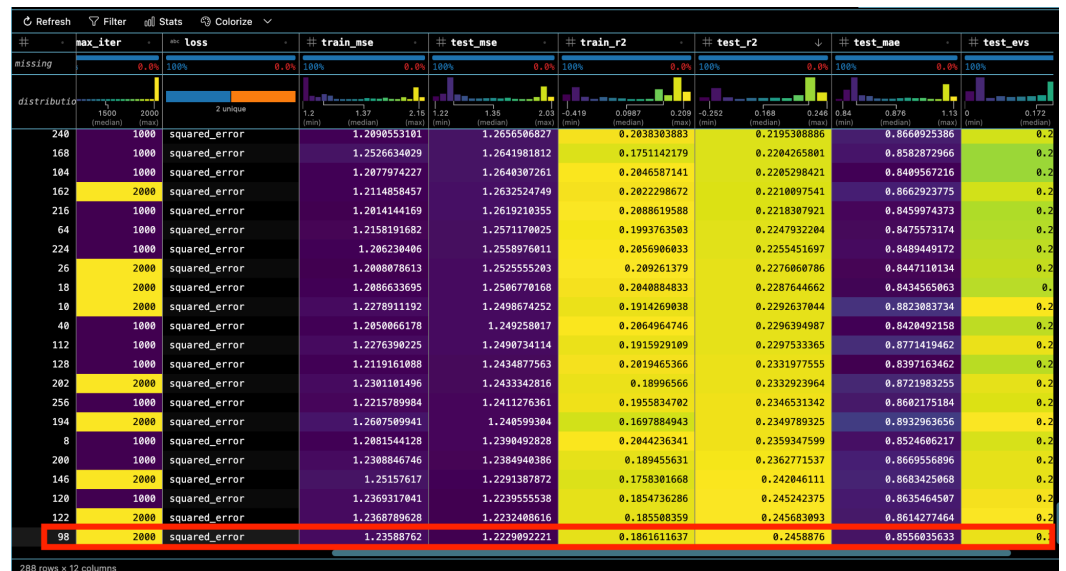


Fig. 1.5.2.

Red Square is Best-Performed

- 1.5.1 After tuning hyperparameters, we concluded that regularization and optimization settings had a measurable effect on model performance across the 288 combinations tested. As shown in *Fig. 1.5.1*, results varied substantially depending on which *penalty*, *alpha*, *learning rate*, *iteration count*, and *loss* function were used. Several combinations, particularly those pairing the *elasticnet* penalty with the *huber* loss at *higher* alpha values, produced *negative* test R^2 scores, meaning those configurations performed worse than simply predicting the average symboling value for every car. This shows that not every hyperparameter combination is appropriate for this dataset, and that some choices actively hurt performance rather than improve it.
- 1.5.2. As seen in *Fig. 1.5.2*, we concluded that **row 98** of the 288 rows tested was the **best-performing** combination. The data shows that the *penalty* was *L2*, *alpha* was *0.0001*, *L1* ratio was *0.15* (not actually applied in this case, since *L1* ratio only affects the *elasticnet* penalty), *learning rate* was *constant*, *max iteration* was *2000*, and *loss* was a *squared error*. This combination produced a test R^2 of *0.246* and a *test MSE* of *1.223*, both improvements over the untuned baseline model's test R^2 of *0.187*, discussed in Section 1.3. This indicates that the hyperparameter tuning meaningfully improved the model's performance, though the improvement was moderate rather than dramatic. The tuned model still explains only about a quarter of the variance in symboling, suggesting that further improvement would likely require additional features or a different modeling approach altogether.

2. Ordinary Linear Square (OLS)

	R2	MSE
Before imputation	0.440	0.821
After imputation	0.611	0.571

- 2.1. When doing OLS regression there aren't any hyper parameters to tune. There is an amount of preprocessing to do on the dataset though. I did the training before and after doing a mean imputation, which means I replaced null values with the average of the columns, and after.
- 2.2. We also notice that as we improve the SGDRegressor algorithm, it approaches the OLS in terms of performance. On a larger dataset the SGDRegressor will happen much faster than SGD given that OLS is $O(n^3)$ and SGD is $O(n)$.
- 2.3. Lastly we notice that the performance on this dataset is extremely poor. The models are limited to linear datasets and as the datasets stray further and further away from being linear, the worse they will perform. Knowing this we can assume that the dataset is non-linear given the performance of the models.